

Model Routing & Selection

Sending each request to the right model — model selection, routing strategies (task, cost, latency, complexity-based), small vs. large models, reasoning vs. chat models, fallback models, and multi-model systems.

DIFFICULTY LEVEL

Intermediate-Advanced

ESTIMATED READING TIME

50-65 minutes

ESTIMATED PRACTICE TIME

80-100 minutes

PREREQUISITES

Modules 01-22, especially 09, 11, 18, 22

LEARNING OUTCOMES

By the end of this module, you will be able to: design a model selection process for a new task; build routing logic based on task type, cost, latency, or complexity; decide when a small vs. large or chat vs. reasoning model fits; design fallback chains; and architect multi-model systems that use different models for different jobs.

Table of Contents

1.	Introduction
2.	Before You Begin
3.	Main Lesson
3.1	Model Selection
3.2	Model Routing
3.3	Task-Based Routing
3.4	Cost-Based & Latency-Based Routing
3.5	Complexity-Based Routing
3.6	Small vs. Large Models
3.7	Reasoning vs. Chat Models
3.8	Fallback Models
3.9	Multi-Model Systems
4.	Visual Learning
5.	Real World Applications
6.	Practical Examples
7.	Code Examples
8.	Common Mistakes
9.	Best Practices
10.	Hands-on Activities
11.	Mini Project
12.	Review Questions
13.	Cheat Sheet
14.	Key Takeaways
15.	Additional Resources

1. Introduction

What is this topic?

This module ties together threads from across the whole course — model comparison (Module 11), cost/latency optimization (Module 09), small models (Module 22) — into a single, practical discipline: deciding, systematically, which model handles which request.

Why is it important?

Most real AI products don't use one single model for everything — they route different requests to different models based on what each request actually needs. Getting this routing right is often the single biggest lever on a system's overall cost, latency, and quality balance.

One Model For Everything

VS.

Right Model For Each Request

Why should AI Engineers learn it?

Always defaulting to the largest, most expensive model is simple but wasteful; always defaulting to the cheapest is simple but risks quality on genuinely hard requests. Deliberate routing is what lets you have both good quality and reasonable cost, rather than picking one.

Where is it used in the real world?

Production AI systems at any real scale almost universally route simple requests to smaller/cheaper models and complex ones to larger/more capable models, often combined with fallback strategies (Module 18, 3.8) for reliability.

Why is it relevant today?

With the growing diversity of available models (Module 11) — across providers, sizes, and specializations (reasoning vs. chat, Module 01, 3.12) — deliberate routing has become a standard, expected part of serious AI system architecture rather than an advanced afterthought.

Note

This module is the natural convergence point of Modules 09 (cost/latency), 11 (model landscape), 18 (fallbacks), and 22 (small models) — if any of those feel unfamiliar, this module will make much more sense after a quick revisit.

2. Before You Begin

RECAP

Reasoning vs. chat models (Module 01, 3.12) — this module's Section 3.7 builds directly on that foundational distinction.

RECAP

Model fallbacks (Module 18, 3.8) — routing and fallback strategy are closely related but distinct: routing chooses proactively based on the request; fallback reacts to a failure.



Tip

Start simple: even a basic "is this a simple lookup or a complex reasoning task?" routing rule captures most of the value before you need anything sophisticated.

3. Main Lesson

Each concept follows: **Definition** → **Simple Explanation** → **Analogy** → **Everyday Example** → **Why It Matters** → **Common Mistakes** → **Best Practices** → **Summary**.

🔗3.1 Model Selection

DEFINITION

Model selection is the deliberate process of choosing which specific model(s) an application will use, based on the task's actual requirements rather than habit or hype.

SIMPLE EXPLANATION

This directly applies Module 11's comparison framework and Module 09's cost/latency concerns to a concrete decision — what capability does this task genuinely need, and which available model(s) meet that bar most efficiently.

💡 *Like choosing the right tool from a well-stocked toolbox for a specific job — not always reaching for the biggest, most powerful tool available, but the one actually suited to the task at hand.*

EVERYDAY EXAMPLE

Selecting a smaller, faster model for a simple intent-classification step and a larger, more capable model for the actual nuanced response generation within the same product.

WHY IT MATTERS

Model selection is the foundational decision that routing (Section 3.2) builds on — you can't route intelligently between models you haven't first deliberately chosen.

COMMON MISTAKES

Selecting a model once at project start and never revisiting the decision as the model landscape (Module 11) and the project's actual needs evolve.

BEST PRACTICES

Base model selection on evaluated evidence (Module 16) against your actual task, not general reputation; revisit periodically as new models become available.

SUMMARY

Model selection is choosing the right model(s) based on genuine task requirements and evidence — the foundation every routing strategy in this module builds on.

🔗3.2 Model Routing

DEFINITION

Model routing is automatically directing each individual request to a specific model (from among several selected options, Section 3.1) based on defined rules or logic, rather than using one fixed model for every request.

SIMPLE EXPLANATION

Where model selection is a one-time architectural decision ("which models will we use"), routing is the ongoing, per-request decision ("which of those models handles this specific request") — the operational layer that makes multi-model systems (Section 3.9) actually work.



💡 *Like a hospital triage nurse directing each incoming patient to the right specialist based on their specific symptoms, rather than sending everyone to the same doctor regardless of need.*

EVERYDAY EXAMPLE

A customer support system routing simple FAQ-style questions to a small, fast model and routing complex, multi-step troubleshooting requests to a larger, more capable model.

WHY IT MATTERS

Routing is what actually captures the cost/quality benefits of having multiple available models — without it, model selection alone doesn't help if you still use only one model for everything.

COMMON MISTAKES

Building an elaborate multi-model architecture but never actually implementing routing logic, defaulting every request to the same model anyway.

BEST PRACTICES

Start with simple, rule-based routing (Sections 3.3-3.5) before considering more sophisticated learned routing approaches.

SUMMARY

Model routing directs each request to the right model per defined logic — the operational mechanism that makes model selection's benefits real in practice.

3.3 Task-Based Routing

DEFINITION

Task-based routing sends requests to different models based on what kind of task they represent — classification, summarization, code generation, creative writing — each potentially suited to a different model.

SIMPLE EXPLANATION

Since different models have different relative strengths (Module 11) — one better at coding, another at multimodal tasks — task-based routing matches each request type to the model best suited for it specifically, rather than using one generalist model for every task type.

 *Like a law firm routing a contract dispute to a contracts specialist and a criminal case to a criminal law specialist, rather than having every case handled by the same generalist attorney.*

EVERYDAY EXAMPLE

An application routing code-generation requests to a model known for strong coding performance (Module 11, 3.2) and creative-writing requests to a different model, based on the task type detected in the request.

WHY IT MATTERS

Task-based routing directly leverages Module 11's model comparison insights operationally — turning "this model is better at X" into an actual routing decision.

COMMON MISTAKES

Routing based on outdated model comparison data — model relative strengths shift over time as providers release updates (Module 11's evergreen warning applies here directly).

BEST PRACTICES

Base task-routing rules on your own current evaluation data (Module 16) for each task type, re-testing periodically as models evolve.

SUMMARY

Task-based routing matches request type to each model's relative strengths — a direct, practical application of Module 11's comparison framework.

3.4 Cost-Based & Latency-Based Routing

DEFINITION

Cost-based routing sends requests to cheaper models when acceptable quality allows; latency-based routing sends requests to faster models when response speed is prioritized over marginal quality gains — both applying Module 09's optimization principles as live routing rules.

SIMPLE EXPLANATION

These routing strategies operationalize Module 09's cost/latency optimization directly into runtime decisions — instead of a one-time model choice, the system dynamically picks based on current priorities (e.g., a premium user tier gets the higher-quality model; a time-sensitive feature gets the faster one).

💡 *Like a shipping service offering both standard and expedited options at different price points — the "routing" decision (which shipping method) depends on what the customer actually needs for that specific package, not a fixed company-wide default.*

EVERYDAY EXAMPLE

A product offering a free tier routed to a smaller, cheaper model and a paid tier routed to a larger, higher-quality model — cost-based routing tied directly to business model.

WHY IT MATTERS

These routing types let a single product serve different quality/cost/speed tradeoffs simultaneously, rather than forcing one fixed tradeoff on every user and use case.

COMMON MISTAKES

Routing purely on cost without any quality floor, risking a genuinely bad experience for requests that needed more capability than the cheap route provided.

BEST PRACTICES

Set a minimum quality floor (validated via evaluation, Module 16) even for cost/latency-optimized routes; monitor (Module 17) whether cheap/fast routes are actually meeting that floor in production.

SUMMARY

Cost-based and latency-based routing operationalize Module 09's optimization principles into live, per-request decisions — powerful, but need a quality floor to stay safe.

🧠 3.5 Complexity-Based Routing

DEFINITION

Complexity-based routing sends requests to different models based on an estimate of how difficult the specific request is, sending simple requests to smaller/cheaper models and hard ones to larger/more capable models.

SIMPLE EXPLANATION

This is often the most impactful routing strategy in practice — a simple, cheap "difficulty classifier" step (sometimes just a small model itself) estimates request complexity before routing, capturing most of the cost savings of small models (Module 22, 3.4) while reserving large-model quality for requests that actually need it.

```
Request → [Complexity Classifier] → Simple? → Small/Fast Model  
                                     → Complex? → Large/Capable Model
```

💡 *Like a hospital's triage system sending routine checkups to a general practitioner and complex, unusual cases to a specialist — most patients get fast, appropriate, lower-cost care, and the specialist's more expensive time is reserved for cases that genuinely need it.*

EVERYDAY EXAMPLE

A customer support system where a lightweight classifier flags "simple FAQ match"

requests for a small model, while anything flagged "unclear" or "multi-step" routes to a larger model.

WHY IT MATTERS

Complexity-based routing is often where the biggest cost savings live — most real-world request volume tends to skew toward simpler cases, meaning a large majority of traffic can often be handled efficiently.

COMMON MISTAKES

Using an unreliable or unvalidated complexity classifier, causing genuinely hard requests to be misrouted to an underpowered model.

BEST PRACTICES

Validate your complexity classifier's accuracy (Module 16) specifically, since misclassification here directly causes quality problems; err toward routing borderline cases to the larger model.

SUMMARY

Complexity-based routing sends easy requests to cheap models and hard ones to capable models — often the single highest-leverage routing strategy for real-world cost savings.

3.6 Small vs. Large Models

DEFINITION

This is the specific routing decision of choosing between a small, efficient model (Module 22, 3.4) and a large, more capable one for a given request or task type.

SIMPLE EXPLANATION

This decision underlies most of Sections 3.3-3.5's routing strategies — the actual choice being routed between is very often exactly this: small vs. large, connecting directly to Module 22's model optimization content.

 *Like deciding whether a specific task needs a specialist consultant or can be handled by a competent generalist — the right answer depends entirely on the specific task, not a fixed rule applied everywhere.*

EVERYDAY EXAMPLE

Using a small model to draft an initial response and a large model only to review/refine it for particularly sensitive or high-stakes cases — a hybrid approach using both at different stages.

WHY IT MATTERS

This is the most fundamental routing decision most systems make — nearly every other routing strategy in this module is, at its core, deciding when to reach for a small vs. large model.

COMMON MISTAKES

Treating this as a one-time, project-wide decision rather than a per-request routing decision that should vary based on task complexity (Section 3.5).

BEST PRACTICES

Default to evaluating small models first for any given task (Module 22, 3.4), reserving large models specifically for cases with evidence they're needed.

SUMMARY

Small vs. large model choice is the most fundamental routing decision — most other routing strategies in this module are ways of making this choice systematically per request.

3.7 Reasoning vs. Chat Models

DEFINITION

This is the routing decision between a fast, standard chat model and a slower, more deliberate reasoning model (Module 01, 3.12), based on whether a request genuinely needs multi-step deliberation.

SIMPLE EXPLANATION

This extends Module 01, 3.12's foundational distinction into an explicit routing rule — a router estimates whether a request needs genuine step-by-step reasoning (math, complex logic, multi-step planning) versus a fast, direct answer, and routes accordingly.

💡 *Like deciding whether a question needs a quick, confident answer off the top of your head or genuinely deserves sitting down with pen and paper to work through carefully — using the slower approach only when the problem actually calls for it.*

EVERYDAY EXAMPLE

Routing a simple factual lookup to a fast chat model, but routing a complex multi-step debugging request to a reasoning-mode model that will work through the problem more carefully.

WHY IT MATTERS

Reasoning modes are typically slower and more expensive (Module 01, 3.12) — routing correctly avoids paying that cost for requests that didn't need it, while still applying it where it genuinely improves accuracy.

COMMON MISTAKES

Defaulting to reasoning mode for every request "to be safe," incurring unnecessary latency and cost on the majority of requests that didn't actually need it.

BEST PRACTICES

Use a lightweight signal (task type, detected complexity, explicit user flag) to route only genuinely reasoning-heavy requests to reasoning mode.

SUMMARY

Reasoning vs. chat routing applies Module 01's foundational model-type distinction as an explicit, request-level routing decision.

✂3.8 Fallback Models

DEFINITION

Fallback models (revisiting Module 18, 3.8 specifically in a routing context) are alternative models a router switches to when the primary chosen route fails, times out, or is unavailable.

SIMPLE EXPLANATION

Where Sections 3.3-3.7's routing strategies are proactive (choosing based on the request itself), fallback routing is reactive (choosing based on a failure) — the two work together as a complete routing system, connecting directly to Module 18's reliability engineering.

💡 *Like a delivery service's primary route being a direct highway, with a pre-planned backup route ready if that highway is unexpectedly closed — proactive planning (which route to take) plus reactive adaptation (what to do if it fails).*

EVERYDAY EXAMPLE

A router that normally sends complex requests to a specific large model, but automatically falls back to an alternative large model from a different provider if the primary one returns errors (Module 18, 3.4).

WHY IT MATTERS

Combining proactive routing with reactive fallback gives a system both efficiency (right model for the job) and resilience (still works if a specific model/provider has issues).

COMMON MISTAKES

Building sophisticated proactive routing logic (Sections 3.3-3.5) with no fallback plan at all for when the chosen route itself fails.

BEST PRACTICES

Design every routing tier with a defined fallback, and test those fallback paths periodically (Module 18, 3.8), not just assume they work.

SUMMARY

Fallback models are the reactive complement to proactive routing — together they provide both efficiency and reliability.

3.9 Multi-Model Systems

DEFINITION

A multi-model system is an AI application architecture that deliberately uses several different models together, each handling the specific role it's best suited for, coordinated by routing logic (Sections 3.2-3.8).

SIMPLE EXPLANATION

This is the architectural outcome of applying every concept in this module together — model selection choosing the roster, routing directing traffic, and fallbacks providing resilience, forming a coherent system rather than a single-model application.

 *Like a well-run hospital rather than a single doctor's office — many specialists (models), a triage/routing system directing patients to the right one, and backup coverage (fallbacks) if a specialist is unavailable, all working together as one coherent system.*

EVERYDAY EXAMPLE

A customer support product using a small model for intent classification, a mid-size model for routine responses, a large model for complex escalations, and a reasoning model specifically for technical troubleshooting — four models, one coherent system.

WHY IT MATTERS

Multi-model systems capture the full benefit of the model landscape's diversity (Module 11) — no single model is optimal for every task, so using several deliberately, well-routed, usually outperforms forcing one model to do everything.

COMMON MISTAKES

Adding model diversity for its own sake without clear routing logic connecting each model to a genuine, evaluated need.

BEST PRACTICES

Design multi-model systems incrementally — start with one model, add a second only when you have clear evidence (Module 16) a specific task would benefit, growing the system deliberately rather than all at once.

SUMMARY

Multi-model systems combine selection, routing, and fallback into one coherent architecture — the natural, evidence-driven endpoint of this module's concepts working together.

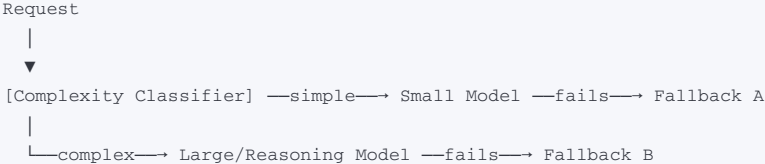
4. Visual Learning

📍 Routing Strategy Family

Routing Strategies

- └─ **Proactive** Task, Cost, Latency, Complexity-based
- └─ **Reactive** Fallback models (Module 18)

🔄 A Complete Routing Flow



📊 Routing Dimension Comparison

Routing Dimension	Optimizes For
Task-based	Matching model strength to task type
Cost-based	Budget/tier-appropriate spending
Latency-based	Response speed priorities
Complexity-based	Overall cost savings at scale

5. Real World Applications

Product	How these fundamentals show up
Freemium AI products	Cost-based routing sends free-tier users to smaller models, paid users to larger ones.
Coding assistants	Complexity-based routing reserves reasoning models for genuinely hard debugging tasks.
Customer support platforms	Multi-model systems combine classification, response generation, and escalation-handling models.
Enterprise AI gateways (Module 18, 3.3)	Centralize routing logic and fallback chains across many internal AI applications.

6. Practical Examples

Beginner: Sketching a routing rule

Write a simple routing rule (in plain English) for a hypothetical support bot: what determines whether a request goes to a small or large model?

Beginner: Fallback chain design

Design a 2-tier fallback chain for a critical feature — primary model, fallback model, and what triggers the switch.

Intermediate: Complexity classifier design

Design (on paper) a lightweight complexity classifier for a customer support use case — what signals would it use to decide "simple" vs. "complex"?

Advanced: Full multi-model architecture

Design a complete multi-model system for a project from Module 14, specifying each model's role, the routing logic between them, and fallback chains.

7. Code Examples

🐍Python — Complexity-based router with fallback

```
# pip install anthropic
import anthropic

client = anthropic.Anthropic(api_key="YOUR_API_KEY")

SMALL_MODEL = "claude-haiku-4-5"
LARGE_MODEL = "claude-sonnet-4-6"
FALLBACK_MODEL = "claude-sonnet-4-6" # could be a different provider (Module 12)

def estimate_complexity(request_text):
    # Simplified heuristic; a real system might use a small classifier model
    complex_signals = ["debug", "compare", "analyze", "why does", "multi-step"]
    return "complex" if any(s in request_text.lower() for s in complex_signals) else "simple"

def route_and_call(request_text):
    complexity = estimate_complexity(request_text)
    model = LARGE_MODEL if complexity == "complex" else SMALL_MODEL

    try:
        response = client.messages.create(
            model=model, max_tokens=300,
            messages=[{"role": "user", "content": request_text}]
        )
        return response.content[0].text, model
    except anthropic.APIError:
        # Reactive fallback (Module 18, 3.8) if the routed model fails
        response = client.messages.create(
            model=FALLBACK_MODEL, max_tokens=300,
            messages=[{"role": "user", "content": request_text}]
        )
        return response.content[0].text, f"{model} → fallback: {FALLBACK_MODEL}"

answer, used_model = route_and_call("Why does my recursive function cause a stack overflow?")
print(f"[{used_model}] {answer}")
```

🌐JavaScript — Cost-tier routing

```
// Route based on user's subscription tier (Module 09 cost tradeoffs, applied per-user)
const MODEL_BY_TIER = {
  free: "claude-haiku-4-5",
  pro: "claude-sonnet-4-6",
  enterprise: "claude-opus-4-8",
};

async function routeByTier(userTier, prompt) {
  const model = MODEL_BY_TIER[userTier] || MODEL_BY_TIER.free;

  const response = await client.messages.create({
    model,
    max_tokens: 300,
    messages: [{ role: "user", content: prompt }],
  });

  return { text: response.content[0].text, modelUsed: model };
}

// Same application logic, different cost/quality tradeoff per user tier
const result = await routeByTier("pro", "Summarize this document...");
```



Tip

Log which model handled each request (as both examples do) — this data feeds directly into Module 17's observability and lets you validate whether your routing rules are actually working as intended in production.

8. Common Mistakes

- **Selecting a model once and never revisiting** as the model landscape and task needs evolve.
- **Building multi-model architecture with no actual routing logic**, defaulting every request to one model anyway.
- **Routing purely on cost with no quality floor**, risking bad experiences on under-routed requests.
- **Using an unvalidated complexity classifier**, misrouting genuinely hard requests to underpowered models.
- **Defaulting to reasoning mode for every request "to be safe,"** incurring unnecessary cost and latency.
- **Building proactive routing with no fallback plan** for when the chosen route itself fails.
- **Adding model diversity without clear, evaluated justification** for each model's specific role.

9. Best Practices

- **Base model selection and routing rules on evaluation evidence** (Module 16), not assumption or habit.
- **Start with simple, rule-based routing** before considering more sophisticated approaches.
- **Set a quality floor** for cost/latency-optimized routes and monitor it in production (Module 17).
- **Validate complexity classifiers specifically**, erring toward the larger model on borderline cases.
- **Pair every proactive route with a defined, tested fallback** (Module 18).
- **Grow multi-model systems incrementally**, adding a new model only with clear evidence of need.

10. Hands-on Activities

Easy 5 exercises

1. Explain, in your own words, the difference between model selection and model routing.
2. Write a simple task-based routing rule for 3 different task types you might encounter in an application.
3. Sketch the complete routing flow diagram from Section 4 from memory.
4. List 2 signals a lightweight complexity classifier might use to flag a request as "hard."
5. Explain why routing purely by cost, with no quality floor, is risky.

Intermediate 3 exercises

1. Build the Python complexity-based router from Section 7 and test it against 10 varied requests, checking whether the routing decisions make sense.
2. Build the JS cost-tier router and simulate requests from all 3 tiers.
3. Design and implement a fallback chain (Module 18-style) on top of your Section 7 router.

Challenge 2 exercises

1. Build a full multi-model system for a project from Module 14 with at least 3 distinct model roles and validated routing logic.
2. Design an evaluation plan (Module 16) specifically for your complexity classifier's routing accuracy — how would you measure whether it's making good decisions?

11. Mini Project

Goal

Build a complete routing layer for a project from Module 14, combining complexity-based routing with fallback, and evaluate whether it actually saves cost without hurting quality.

Requirements

- An Anthropic API key
- Python or JavaScript
- A project or task from Module 14/16 with an existing evaluation set

Step-by-Step Instructions

1. Build a simple complexity classifier (heuristic-based is fine to start) for your task.
2. Implement routing: simple requests → small model, complex requests → large model.
3. Add a fallback for each route (Module 18's pattern).
4. Run your full evaluation set (Module 16) through the routed system and compare against always using the large model alone.
5. Compare: overall quality score, total token cost, and average latency between "always large" and "routed."

Expected Output

```
Always-large baseline: Quality 4.4/5, Cost $0.021/req avg, Latency 1.8s avg
Routed system:         Quality 4.3/5, Cost $0.009/req avg, Latency 1.1s avg
```

```
Verdict: Routing achieved ~57% cost reduction and improved
latency with a negligible (0.1) quality difference — clear win
for this task's request distribution.
```

Possible Improvements

- Replace the heuristic classifier with a small dedicated classification model for more reliable routing.
- Add cost-tier routing on top of complexity routing for a full multi-dimensional system.
- Connect routing decisions to Module 17's tracing to monitor routing accuracy over time in production.

12. Review Questions

Multiple Choice (10)

1. Model selection differs from model routing by:

- A) Being the same thing B) Being a one-time architectural choice vs. an ongoing per-request decision C) Never involving evaluation D) Only applying to closed APIs

2. Task-based routing matches requests to models based on:

- A) Random assignment B) Each model's relative strengths for that task type C) Alphabetical order D) Token count only

3. Cost-based routing without a quality floor risks:

- A) Nothing B) Poor experiences on requests under-served by the cheap route C) Higher cost always D) Better quality automatically

4. Complexity-based routing is often described as:

- A) The least impactful routing strategy B) Often the highest-leverage strategy for real-world cost savings C) Irrelevant to cost D) Only useful for image tasks

5. Reasoning vs. chat routing decides based on:

- A) Random chance B) Whether a request genuinely needs multi-step deliberation C) The time of day D) Token pricing only

6. Fallback models are best described as:

- A) Proactive routing B) Reactive routing triggered by a failure C) Unrelated to routing D) Always the same as the primary model

7. A multi-model system combines:

- A) Nothing meaningful B) Model selection, routing, and fallback into one coherent architecture C) Only one model D) Only fallback, no routing

8. Why validate a complexity classifier specifically?

- A) It's unnecessary B) Misclassification directly causes quality problems from misrouting C) It has no effect on quality D) Classifiers are always perfect

9. Why grow multi-model systems incrementally?

- A) It's slower for no reason B) Each added model should have clear, evaluated justification C) More models are always better regardless of evidence D) It's required by every API

10. Defaulting to reasoning mode for every request typically causes:

- A) Lower cost and latency B) Unnecessary cost and latency on requests that didn't need it C) No effect at all D) Better routing automatically

Identification (5)

1. _____ is the deliberate process of choosing which model(s) an application will use.

2. _____ automatically directs each request to a specific model based on defined logic.

3. _____ sends simple requests to smaller models and hard requests to larger ones.

4. _____ are alternative models used when the primary chosen route fails.

5. _____ is an architecture deliberately using several different models together, each in the role they're best suited for.

True or False (5)

1. Model selection and model routing are exactly the same thing.

2. Complexity-based routing is often the highest-leverage routing strategy for cost savings.

3. Cost-based routing should generally include a minimum quality floor.

4. Fallback models are a proactive, request-based routing strategy.

5. Multi-model systems should generally be grown incrementally, with evidence for each addition.

Situational (5)

Situational (5)

- 1. Your product has free and paid tiers with different budget constraints. What routing strategy fits this directly?**
- 2. You want to reserve your most expensive model for only genuinely hard requests. What routing strategy fits?**
- 3. Your primary model provider had a brief outage and your product went down with it. What routing layer was missing?**
- 4. You're building a system with 4 different models each handling a distinct role. What architecture category is this?**
- 5. Your complexity classifier keeps misrouting hard requests to your small model, hurting quality. What should you do?**

Answer Key

Multiple Choice: 1-B, 2-B, 3-B, 4-B, 5-B, 6-B, 7-B, 8-B, 9-B, 10-B

Identification: 1-Model selection, 2-Model routing, 3-Complexity-based routing, 4-Fallback models, 5-Multi-model system

True or False: 1-False, 2-True, 3-True, 4-False, 5-True

Situational (sample reasoning):

1. Cost-based routing.
2. Complexity-based routing.
3. A fallback model/provider (Module 18, 3.8) for the routing layer.
4. A multi-model system.
5. Re-validate and tighten the complexity classifier (Module 16), erring toward the larger model on borderline cases.

13. Cheat Sheet

Selection vs. Routing

Selection — one-time, which models to use.
Routing — per-request, which model handles this.

Key Choices

Small vs. large, reasoning vs. chat — the core decisions most routing logic makes.

Routing Dimensions

Task type, cost tier, latency need, complexity — often combined.

Complete System

Proactive routing + reactive fallback = a full multi-model system.

14. Key Takeaways

- Model selection is a one-time architectural choice; model routing is the ongoing, per-request decision that makes selection's benefits real.
- Task, cost, latency, and complexity-based routing each optimize for a different dimension — often combined in practice.
- Complexity-based routing is often the single highest-leverage strategy for real-world cost savings.
- Small-vs-large and reasoning-vs-chat are the two most fundamental routing decisions underlying most routing systems.
- Fallback models provide the reactive complement to proactive routing — both are needed for a resilient system.
- Multi-model systems combine selection, routing, and fallback into one coherent architecture, grown incrementally with evidence.

15. Additional Resources

Related Modules

- Module 09 — Prompt Optimization (cost/latency fundamentals routing operationalizes)
- Module 11 — Model Comparison (the landscape routing decisions draw from)
- Module 18 — LLMOps & Production AI (fallback strategy in depth)
- Module 22 — Model Optimization (small models as a routing target)

Tools

- LLM routing/gateway platforms (e.g., LiteLLM, OpenRouter's routing features, Module 12) for pre-built multi-provider routing infrastructure.

Communities & Further Learning

- AI infrastructure engineering blogs covering production routing architectures at scale.